

网络空间安全实验基础

Hands-on Cybersecurity – Fundamentals

公钥基础设施（PKI）

主讲人：张玉健



课程提纲

- 中间人（Man-In-The-Middle）攻击
- 公钥基础设施
- 数字证书：X. 509
- 如何获得和使用数字证书
- PKI 面临的安全风险



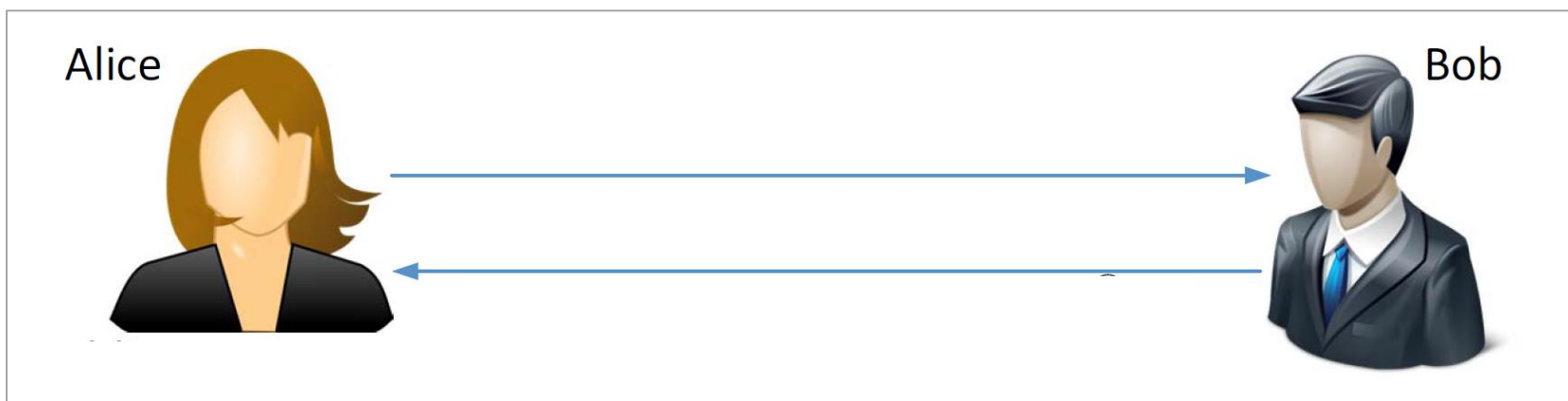
课程提纲

- 中间人（Man-In-The-Middle）攻击
- 公钥基础设施
- 数字证书：X. 509
- 如何获得和使用数字证书
- PKI 面临的安全风险



互联网的基本需求

两方通信



- Alice、Bob可以是个人（PC）、机构（Website）等。
- 在开放的网络环境中，面临风险：偷听（Eavesdropping）、中间人（Man-In-The-Middle）攻击等

缓解措施：加密

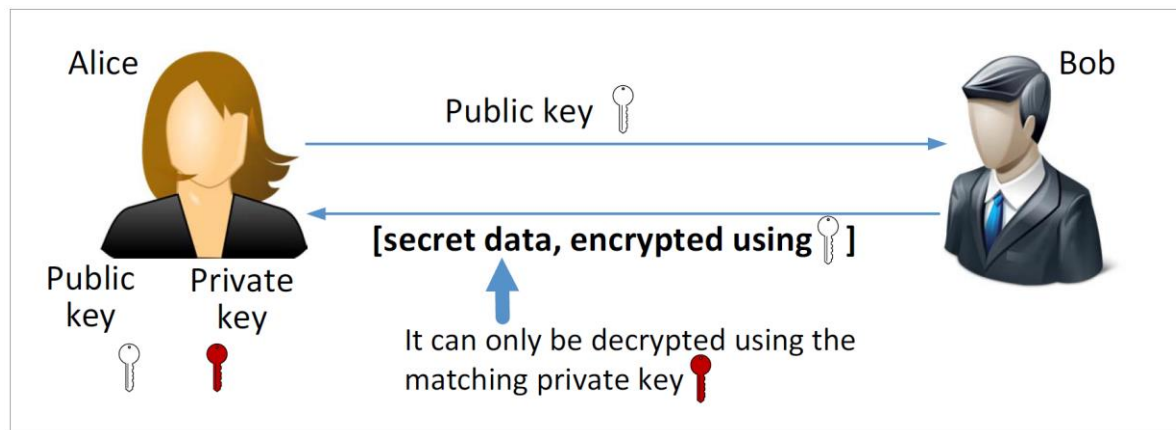


存在的问题

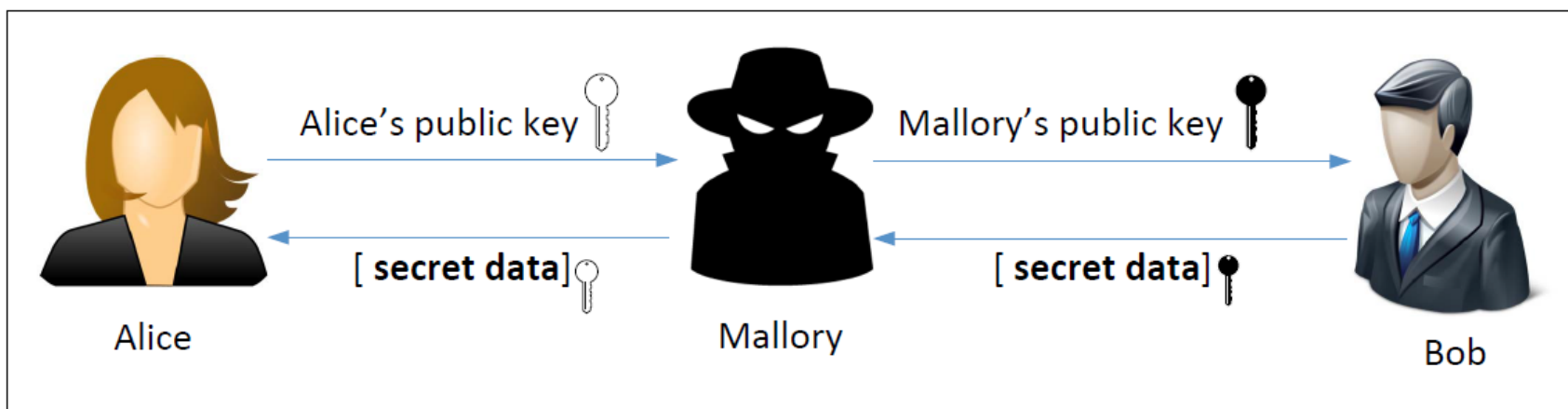
- 加密算法：对称加密 v. s. 非对称加密（公钥加密） ➡ 混合 { 正常会话：对称加密（如AES）
密钥协商：公钥加密（如RSA）
- 密钥交换：如何安全的进行密钥交换？ ➡ 会话密钥 → 公钥加密 → ?

公钥加密的中间人攻击

你以为



实际上

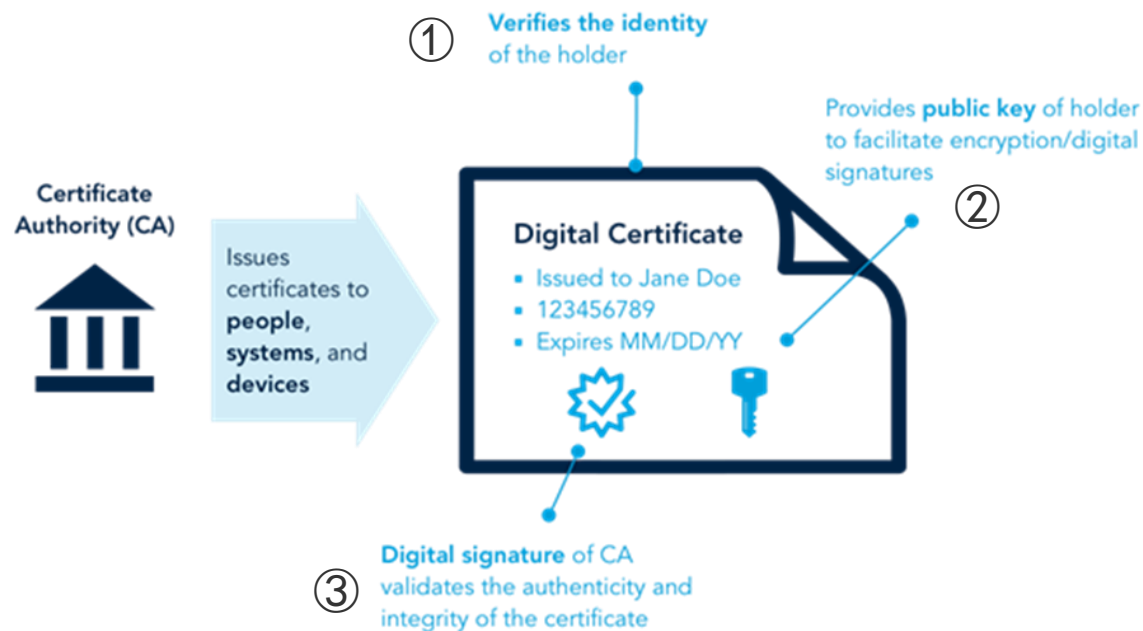


Why? : 没有验证Alice与Alice's public key的绑定关系

解决方案

验证Alice与Alice' s public key的绑定关系

- ① 引入可信第三方验证ID
- ② 将ID与Public key绑定在证书中
- ③ 可信第三方对其进行签名（防止证书被篡改或伪造）



课程提纲

- 中间人（Man-In-The-Middle）攻击
- 公钥基础设施
- 数字证书：X.509
- 如何获得和使用数字证书
- PKI 面临的安全风险



一个比喻

你的身份与某项技能的绑定关系，例如驾照

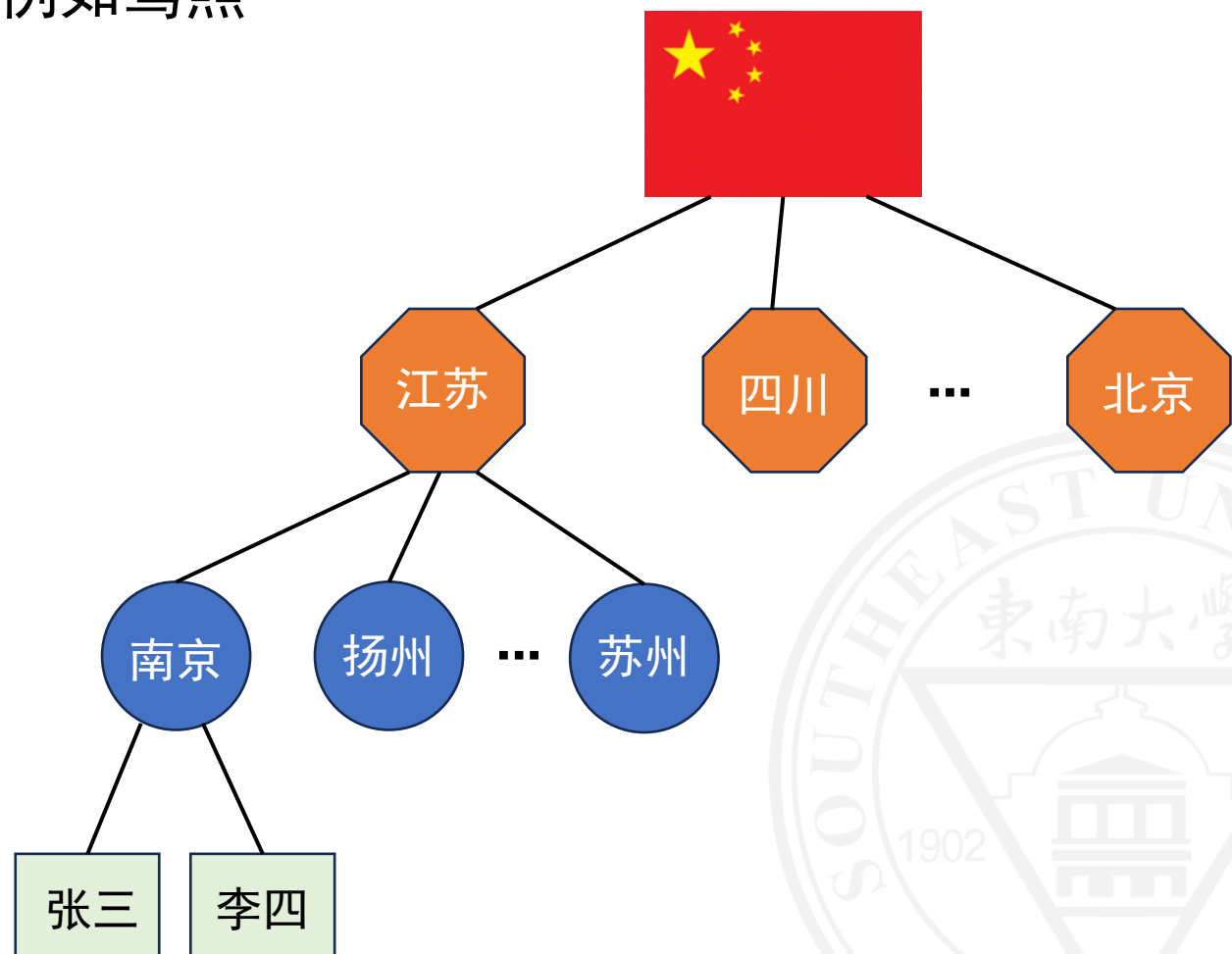


各地颁发、全国互认



当场测试（认证）

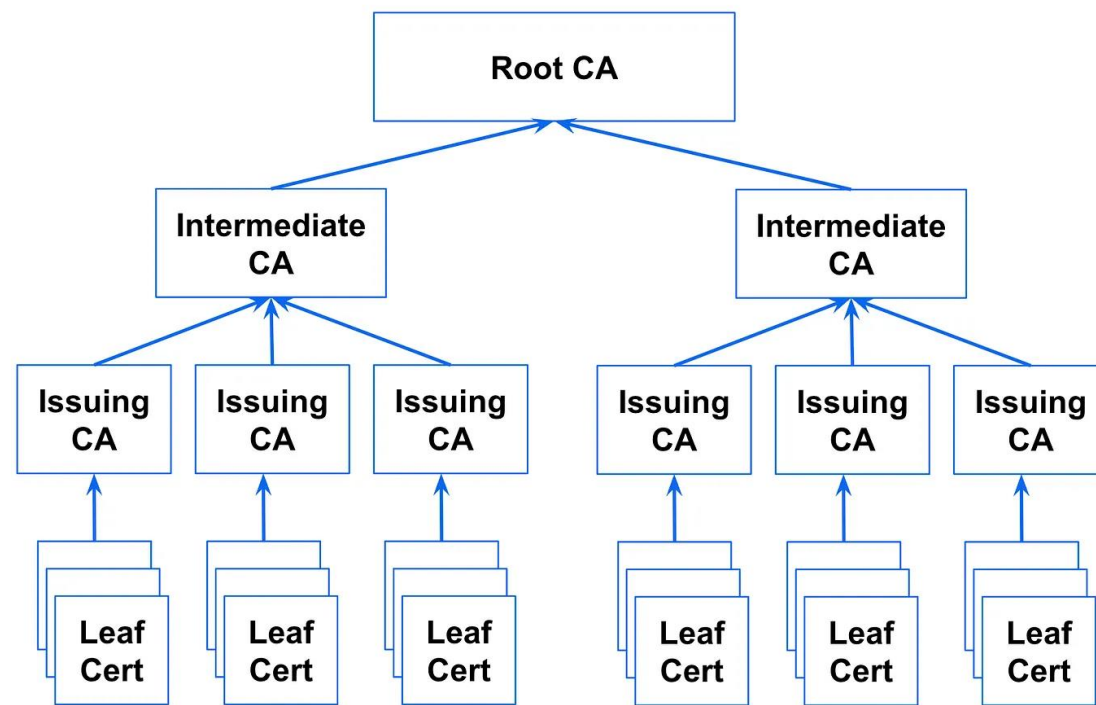
看驾照



PKI的层次结构

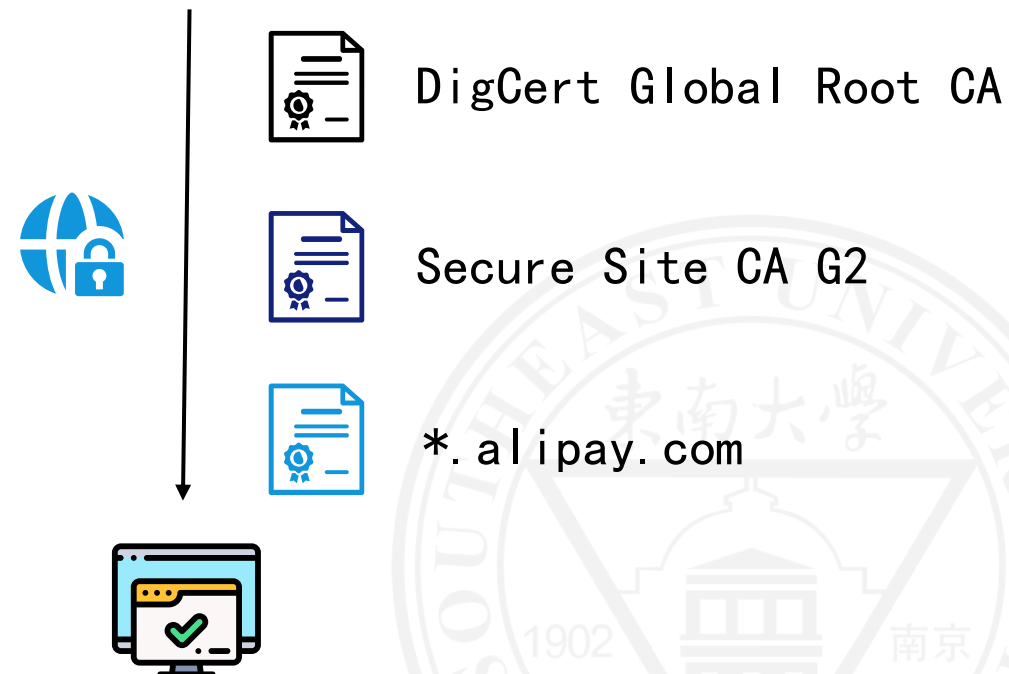
➤ 重要概念

- 证书 (Certificate)：一个数字文件 (X.509标准)，主要包含证书所有者的信息、证书所有者的公钥和证书认证机构的签名等内容。
- 证书认证机构 (Certificate Authority, CA)：一个可信的权威机构，负责验证用户身份，并为其颁发或吊销证书。
 - ✓ 根证书机构 (Root CA)：证书的信任根。
 - ✓ 中间CA (Intermediate CA)：根证书机构的代理，可以有多级。
 - ✓ 签发CA (Issuing CA)：直接颁发数字证书的CA。



感受下现实中的例子（1/2）

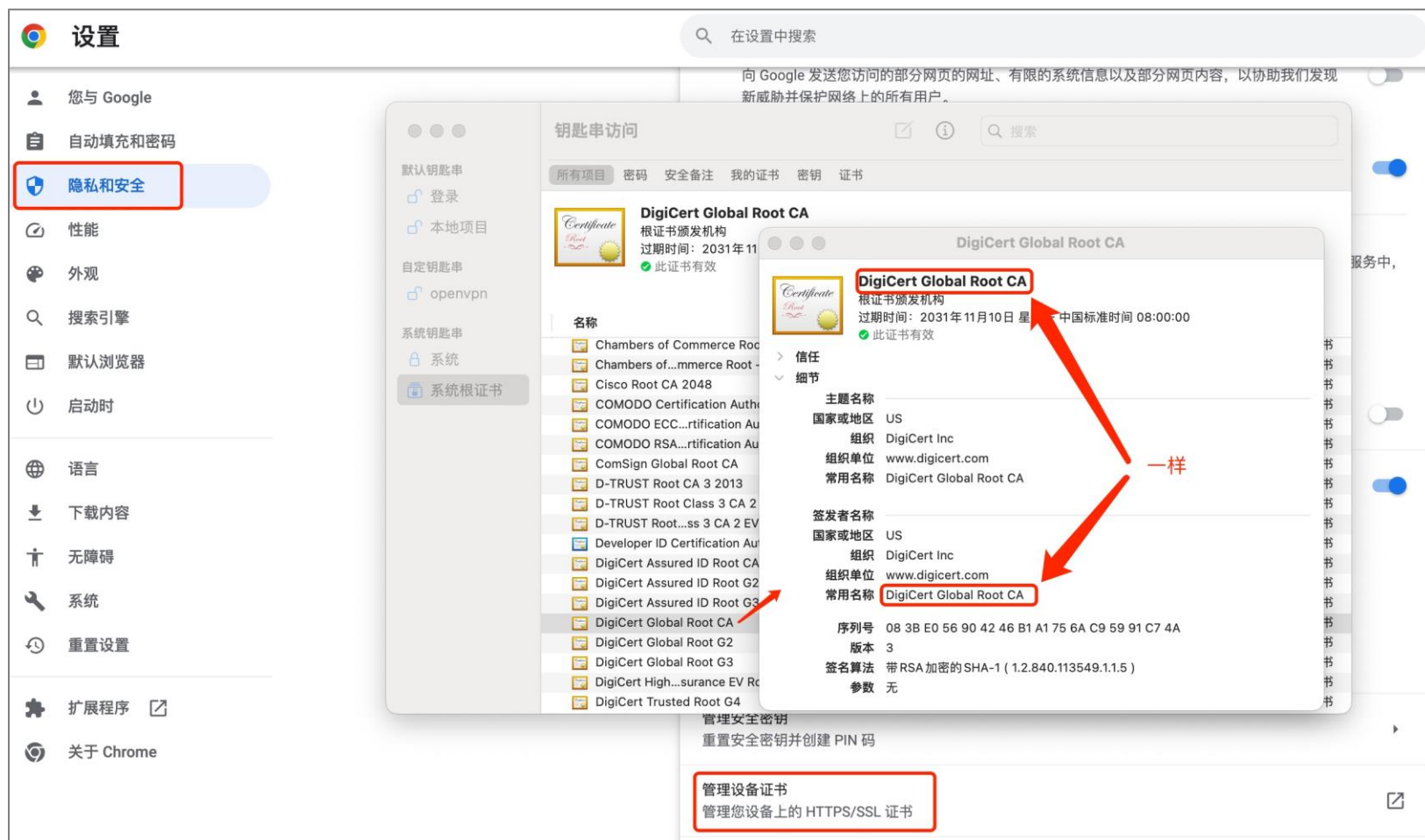
用浏览器（如Chrome）打开带有证书的网站（如https://www.alipay.com）



问题：DigiCert Global Root CA由谁背书？

感受下现实中的例子（2/2）

打开浏览器（如Chrome）的根证书列表寻找DigiCert Global Root CA的证书



根证书

- 由Root CA自签
- 操作系统、浏览器厂商负责对其验证并集成

Root of Trust

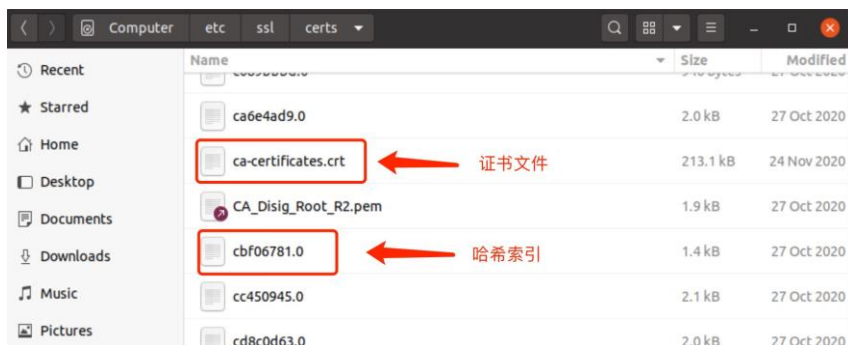
如何找到根证书

➤ 根证书的位置

- **操作系统**：存放于文件目录，例如Linux一般在/etc/ssl/certs/
- **浏览器**：相关配置下
 - ✓ Chrome: Settings -> Show advanced settings -> Manage Certificates
 - ✓ Firefox: Edit -> Preferences -> Advanced -> Certificates -> View Certificates -> Certificate Manager -> Authorities

➤ 程序的查找方式

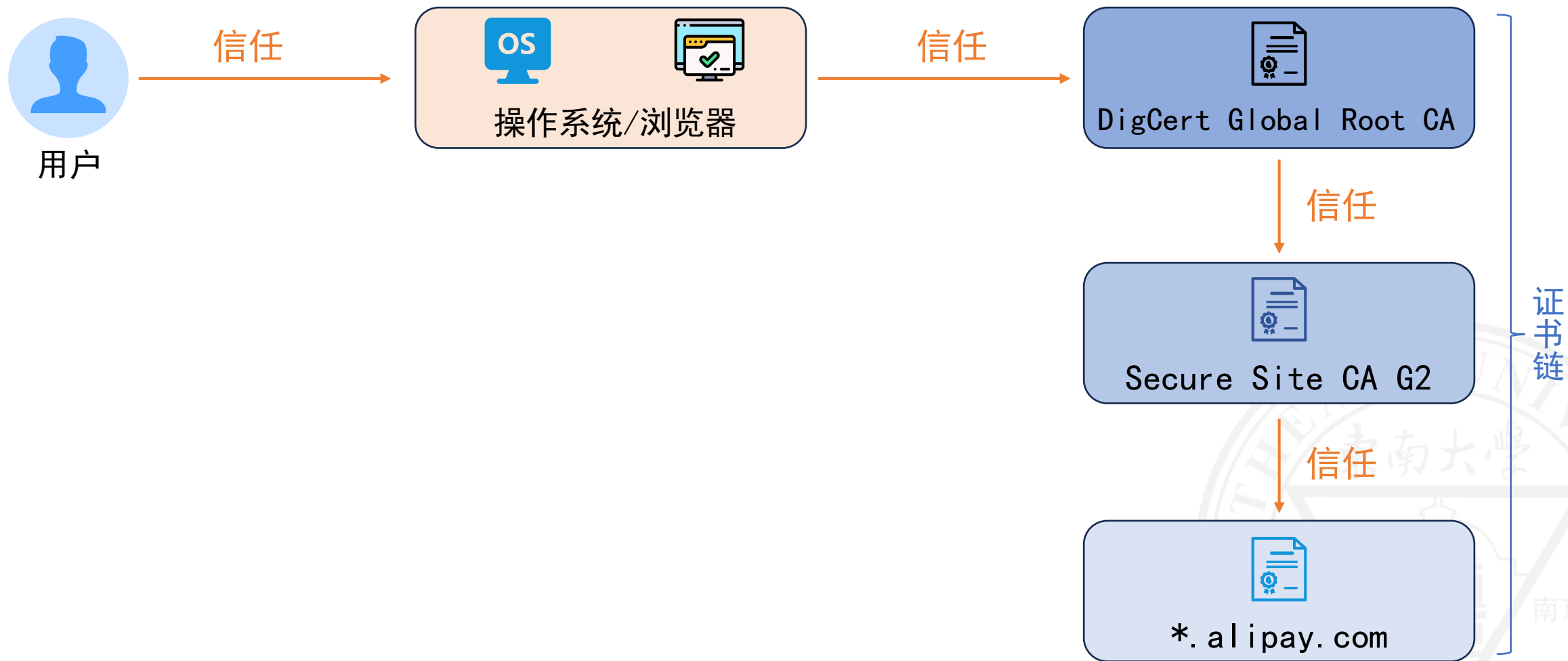
- **哈希索引**：预先计算证书文件的Hash值，使用时通过哈希值找到证书



可用命令计算哈希索引

```
$ openssl x509 -in someCA.crt -noout -subject -hash
```

PKI 的信任链



课程提纲

- 中间人（Man-In-The-Middle）攻击
- 公钥基础设施
- **数字证书：X.509**
- 如何获得和使用数字证书
- PKI 面临的安全风险



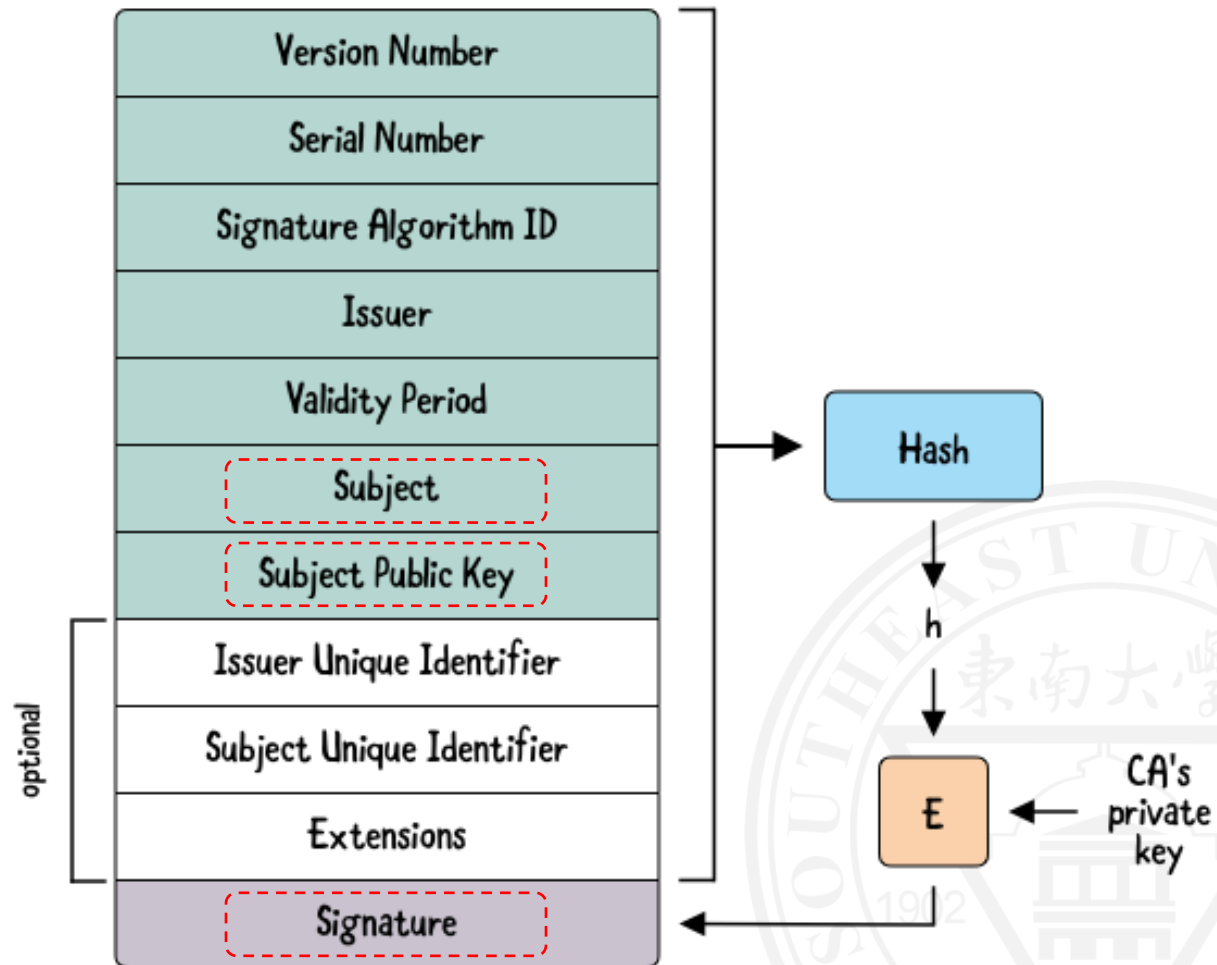
X.509数字证书

➤ 必要信息

- 公钥 (Public key)
- 公钥所有者 (Owner of the public key)
- CA的签名 (CA's signature)

➤ X.509

- 概念：公钥证书的格式标准。



案例分析：Alipay的证书（1/4）

➤ 获取Alipay的证书链

```
$ openssl s_client -showcerts -connect www.alipay.com:443
```

```
Certificate chain
0 s:C = CN, ST = Shanghai, O = "Alipay.com Co.,Ltd", CN = *.alipay.com
  i:C = US, O = DigiCert Inc, OU = www.digicert.com, CN = Secure Site CA G2
-----BEGIN CERTIFICATE-----
MIIJwDCCCKigAwIBAgIQD6F0f9P1Lasc64dPq0r4jjANBgkqhkiG9w0BAQsFADBb
MQswCQYDVQQGEwJVUzEVMBMGA1UEChMMRGlnaUNlcnQgSW5jMRkwFwYDVQQLExB3
d3cuZGlnaWlnaUNlcnQyY29tMR0wGAYDVQQDEXFtZW50cmUgU2l0ZSB0QSBHMjAeFw0y
...
BeHhDAV2P45BCRxLR6paSiEtQ4eL0H8PdsuphXsidnLTF85nS2dddC1yeR8zm+Om
7+prMA==
-----END CERTIFICATE-----
1 s:C = US, O = DigiCert Inc, OU = www.digicert.com, CN = Secure Site CA G2
  i:C = US, O = DigiCert Inc, OU = www.digicert.com, CN = DigiCert Global Root CA
-----BEGIN CERTIFICATE-----
MIIFjCCA06gAwIBAgIQCH4Y+4+qkn7odgoNiYL1EjANBgkqhkiG9w0BAQsFADBh
MQswCQYDVQQGEwJVUzEVMBMGA1UEChMMRGlnaUNlcnQgSW5jMRkwFwYDVQQLExB3
d3cuZGlnaWlnaUNlcnQyY29tMSAwHgYDVQQDEXdEaWdpQ2VydCBHbG9iYWwgUm9vdCBD
...
mycTqbSozQwiiEfb+cxzY82YaNzaLpJyIst0T2QmdDDngmyd2LEmm4NKeXRrcFRh
XDDFfpIn93B7JA==
-----END CERTIFICATE-----
2 s:C = US, O = DigiCert Inc, OU = www.digicert.com, CN = DigiCert Global Root CA
  i:C = IE, O = Baltimore, OU = CyberTrust, CN = Baltimore CyberTrust Root
-----BEGIN CERTIFICATE-----
MIIEYDCCA0igAwIBAgIQD1vDoXbLeJ4gIMeJPIFntDANBgkqhkiG9w0BAQsFADBa
MQswCQYDVQQGEwJJRTESMBAGA1UEChMJQmFsdGltb3JlMRMwEQYDVQQLExwDeWJl
c3RydXN0MSIwIAYDVQQDEXlCYWx0aW1vcmUgU3liZXJUCnVzdCBSc290MB4XDTE2
...
dQt6sFgaFjiUki4bSNrRevngFmMSUYGMkLGELj//so6ofkw4av9cXBZYqIX/XcCj
+KgVQK8z7A0yUhkfMgnzZT6StIQ=
-----END CERTIFICATE-----
```



*.alipay.com

保存



alipay.crt



Secure Site CA G2



DigCert Global Root CA

案例分析：Alipay的证书（2/4）

➤ 解析Alipay的证书

```
$ openssl x509 -in alipay.crt -text -noout
```

```
Certificate:
Data:
  Version: 3 (0x2)
  Serial Number:
    0f:a1:74:7f:d3:f5:2d:ab:1c:eb:87:4f:a8:ea:f8:8e
  Signature Algorithm: sha256WithRSAEncryption
  Issuer: C = US, O = DigiCert Inc, OU = www.digicert.com, CN = Secure Site CA G2
  Validity
    Not Before: Aug  4 00:00:00 2023 GMT
    Not After : Sep  3 23:59:59 2024 GMT
  Subject: C = CN, ST = Shanghai, O = "Alipay.com Co.,Ltd", CN = *.alipay.com
  Subject Public Key Info:
    Public Key Algorithm: rsaEncryption
    RSA Public-Key: (2048 bit)
    Modulus:
      00:a2:75:a6:07:79:3e:78:61:b2:7c:81:dc:0f:b8:
      3a:9c:34:eb:93:77:68:b8:df:47:08:6a:67:61:8a:
      78:52:e4:6e:a3:a1:b8:33:a6:84:e1:f4:3b:ed:c8:
      12:33:46:79:2e:cc:47:15:0f:05:5c:ec:0c:c1:d8:
      f9:c5:a8:80:51:40:2a:74:b8:d9:2a:90:1a:99:b2:
      ab:55:4f:4d:c8:5f:09:61:c0:1a:97:5c:5c:36:5a:
      49:aa:ec:67:c4:6d:b9:03:9a:49:09:e1:6f:1e:56:
      d7:ac:b8:d7:5f:1f:11:1c:26:0d:77:e1:81:2a:b0:
      d3:df:a8:26:3d:f8:dc:cc:88:bd:be:03:85:66:0e:
      a0:87:b5:4e:2c:3f:32:78:1e:3a:cb:97:f4:8a:bd:
      27:87:40:25:57:1e:d7:d9:24:02:01:06:94:96:df:
      62:24:65:1c:77:f8:9d:7c:94:f1:9d:1c:2e:dc:30:
      83:69:f5:fc:e4:a3:86:56:e1:98:97:9a:13:15:84:
      c2:31:39:16:bc:ce:5c:28:77:2b:46:fe:51:73:7d:
      39:32:a0:d0:29:91:27:03:a1:64:ab:cc:19:13:19:
      e4:b9:a9:6c:35:cc:30:a3:45:5e:d1:4b:02:26:35:
      f4:ef:b8:df:fe:dd:c1:2f:a3:11:1c:74:74:00:3e:
      38:6d
    Exponent: 65537 (0x10001)
```

证书序列号

签名算法

证书颁发者的信息

有效期

证书所有者的信息

证书所有者的公钥



案例分析：Alipay的证书（3/4）

```
X509v3 extensions:
X509v3 Authority Key Identifier:
  keyid:C4:11:7E:88:40:86:C2:41:BF:65:F3:1A:E1:B4:53:40:A3:AB:EC:7D

X509v3 Subject Key Identifier:
  78:F5:75:02:C8:66:09:90:3D:5A:C5:81:85:DB:FF:0F:89:87:6F:8D
X509v3 Subject Alternative Name:
  DNS:*.alipay.com, DNS:alipay.com, DNS:*.alipay-cloud.com, DNS:alipay-cloud.com,
  ...
  DNS:alipay.com.cn, DNS:*.open.alipay.com.cn, DNS:open.alipay.com.cn
X509v3 Key Usage: critical
  Digital Signature, Key Encipherment
X509v3 Extended Key Usage:
  TLS Web Server Authentication, TLS Web Client Authentication
X509v3 CRL Distribution Points:

  Full Name:
    URI:http://crl3.digicert.com/SecureSiteCAG2.crl

  Full Name:
    URI:http://crl4.digicert.com/SecureSiteCAG2.crl

X509v3 Certificate Policies:
  Policy: 2.23.140.1.2.2
  CPS: http://www.digicert.com/CPS

Authority Information Access:
  OCSP - URI:http://ocsp.dccocsp.cn
  CA Issuers - URI:http://crl.digicert-cn.com/SecureSiteCAG2.crt

X509v3 Basic Constraints:
  CA:FALSE
CT Precertificate SCTs:
  Signed Certificate Timestamp:
    Version   : v1 (0x0)
    Log ID    : 76:FF:88:3F:0A:B6:FB:95:51:C2:61:CC:F5:87:BA:34:
               B4:A4:CD:BB:29:DC:68:42:0A:9F:E6:67:4C:5A:3A:74
    Timestamp : Aug  4 07:34:10.583 2023 GMT
    Extensions: none
    Signature  : ecdsa-with-SHA256
               30:44:02:20:39:C8:2D:B9:F5:D6:E5:8A:35:74:7A:E4:
               7D:C7:07:49:15:B0:BB:31:2C:DF:49:63:90:25:32:80:
               32:B7:95:02:02:20:68:A7:4F:4B:EA:EA:DB:45:F0:E9:
               8F:DF:F1:62:F1:90:39:5D:D2:A9:BE:3E:CF:B8:D7:80:
               0B:84:5C:9A:C4:88
```

证书所有者别名（CN只有一个，别名可有多个）

证书吊销列表

CA:FALSE表明非CA证书（若为TRUE，则为CA）

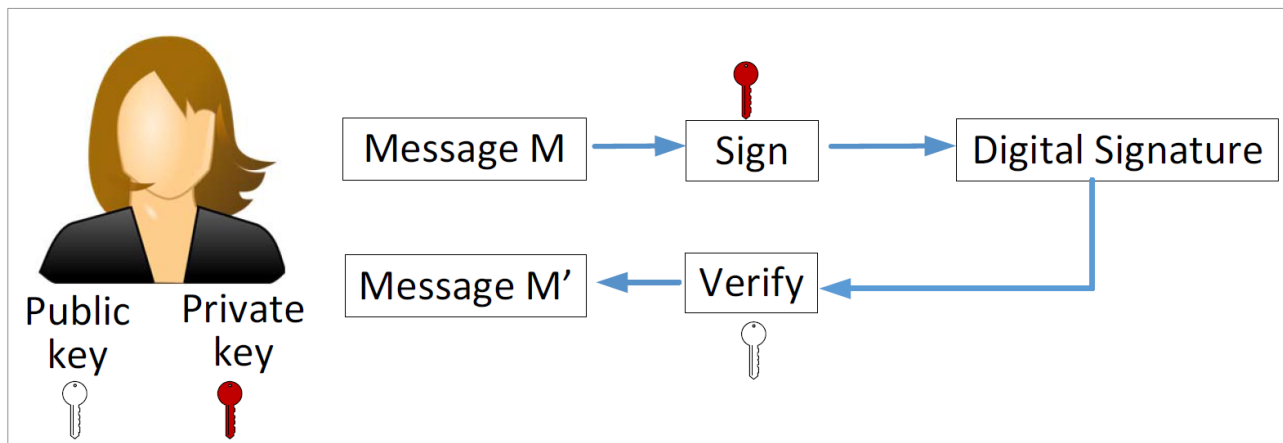
证书透明（Certificate Transparency, CT）

案例分析：Alipay的证书（4/4）

Signature Algorithm: sha256WithRSAEncryption

```
4d:64:11:54:f8:cb:bb:61:48:4b:5f:00:bd:7c:04:65:51:7b:
0e:1a:1e:86:26:ce:67:57:2e:71:ed:c2:ed:07:86:33:9c:e8:
4c:77:60:c6:cf:d1:7e:20:f9:9b:a6:fd:4e:fe:22:9d:9d:21:
a0:2c:ab:4b:fd:bd:31:43:c0:0d:95:24:58:d4:62:dd:96:ea:
c8:68:58:72:c5:0e:f6:7b:1f:cf:74:7a:97:15:66:b0:8d:94:
0c:f7:97:dd:a1:c0:10:d9:08:af:2f:13:6b:18:9c:82:9a:33:
34:c7:8e:b4:20:dc:ac:f9:90:66:a5:15:5f:a4:38:8b:5b:67:
ea:2a:55:d9:df:49:3e:5c:f0:0c:52:45:25:ff:f5:7c:2d:5a:
2c:13:63:47:d8:62:7b:d3:91:2d:ea:bf:8d:58:b1:d4:11:07:
cb:62:74:3e:66:c3:87:ad:29:21:1e:52:cc:74:fe:b2:b8:e9:
31:1a:2b:59:b2:bb:96:ac:6b:cd:51:b4:48:40:f1:76:e8:40:
7a:49:5d:4c:f4:1f:05:e1:e1:0c:05:76:3f:8e:41:09:1c:4b:
47:aa:5a:4a:21:2d:43:87:8b:d0:7f:0f:76:ca:6e:85:7b:22:
76:72:d3:17:ce:67:4b:67:5d:74:2d:72:79:1f:33:9b:e3:a6:
ef:ea:6b:30
```

CA的签名（对前文内容Hash值的签名）



数字签名

- 签名者用其私钥对消息M进行加密；
- 验证者用签名者公钥对M的加密值解密，并与接收到的消息M'进行比较，若一致则验签成功。

课程提纲

- 中间人（Man-In-The-Middle）攻击
- 公钥基础设施
- 数字证书：X. 509
- 如何获得和使用数字证书
- PKI 面临的安全风险



获得及使用证书的步骤

➤ 寻找一个CA

- PKI 中的签发CA
- 自己成为一个CA

➤ 生成证书签名请求（CSR）

➤ CA验证CSR文件

- DV: Domain Validated
- OV: Organization Validated
- EV: Extended Validated

➤ CA用其私钥签名生成证书

➤ 在服务器上部署证书

➤ 浏览器验证证书



成为一个CA（实验任务一）



➤ 创建拥有根证书的CA

• 实验步骤

① 在虚拟机VM的当前实验目录下创建CA的工作目录

```
mkdir -p demoCA && cd demoCA
mkdir -p certs crt newcerts
touch index.txt serial
echo 1000 > serial
cd ..
```

② 创建CA的公私钥并自签证书（将使用默认配置文件/usr/lib/ssl/openssl.cnf）

```
openssl req -x509 -newkey rsa:4096 -sha256 -days 3650 \
    -keyout ca.key -out ca.crt \
    -subj "/CN=www.modelCA.com/O=Model CA LTD./C=US" \
    -passout pass:dees
```

用dees对私钥进行加密

③ 查看证书文件ca.crt和私钥文件ca.key

```
openssl x509 -in ca.crt -text -noout
openssl rsa -in ca.key -text -noout
```

• 要求：记录程序、操作命令和输出结果，写入实验报告，对实验现象进行解释。

创建一个证书签名请求（实验任务二）



➤ 为一个域名创建证书签名请求（Certificate Signing Request, CSR）

• 实验步骤

① 创建www.bank32.com的公私钥和CSR，并添加别名

```
openssl req -newkey rsa:2048 -sha256 \  
            -keyout server.key -out server.csr \  
            -subj "/CN=www.bank32.com/O=Bank32 Inc./C=US" \  
            -passout pass:dees \  
            -addext "subjectAltName = DNS:www.bank32.com, \  
                    DNS:www.bank32A.com, \  
                    DNS:www.bank32B.com"
```

② 查看CSR文件server.csr和私钥文件server.key

```
openssl req -in server.csr -text -noout  
openssl rsa -in server.key -text -noout
```

- 要求：记录程序、操作命令和输出结果，写入实验报告，对实验现象进行解释。



为你的域名生成证书（实验任务三）



➤ 生成域名的数字证书

• 实验步骤

① 修改默认配置文件（可在拷贝版上修改）

```
cp /usr/lib/ssl/openssl.cnf myCA_openssl.cnf  
vi myCA_openssl.cnf
```

去掉copy_extensions选项前的注释，使得颁发证书时能够复制extension字段

```
# Extension copying option: use with caution.  
copy_extensions = copy
```

② 生成证书文件server.crt

```
openssl ca -config myCA_openssl.cnf -policy policy_anything \  
-md sha256 -days 3650 \  
-in server.csr -out server.crt -batch \  
-cert ca.crt -keyfile ca.key
```

③ 查看证书文件

```
openssl x509 -in server.crt -text -noout
```

- 要求：记录程序、操作命令和输出结果，写入实验报告，对实验现象进行解释。



为你的网站添加证书（实验任务四-1/3）



➤ 在Apache服务器上部署刚刚生成的数字证书

• 实验步骤

① 在虚拟机VM上当前实验目录的Labsetup下，启动容器

```
docker-compose build
docker-compose up
```



```
[08/19/23] seed@VM:~/.../Labsetup$ docker-compose up
Creating network "net-10.9.0.0" with the default driver
Creating www-10.9.0.80 ... done
Attaching to www-10.9.0.80
```

② 进入容器，查看Apache相关配置文件

```
[08/19/23] seed@VM:~/.../Public-Key Infrastructure (PKI) Lab$ dockps
6ef099bd1393 www-10.9.0.80
[08/19/23] seed@VM:~/.../Public-Key Infrastructure (PKI) Lab$ docksh 6ef099bd1393
root@6ef099bd1393:/# more /etc/apache2/sites-available/bank32_apache_ssl.conf
<VirtualHost *:443>
    DocumentRoot /var/www/bank32
    ServerName www.bank32.com
    ServerAlias www.bank32A.com
    ServerAlias www.bank32B.com
    ServerAlias www.bank32W.com
    DirectoryIndex index.html
    SSLEngine On
    SSLCertificateFile /certs/bank32.crt
    SSLCertificateKeyFile /certs/bank32.key
</VirtualHost>

<VirtualHost *:80>
    DocumentRoot /var/www/bank32
    ServerName www.bank32.com
    DirectoryIndex index_red.html
</VirtualHost>

# Set the following gloal entry to suppress an annoying warning message
ServerName localhost
```

HTTPS

HTTP

相关目录

- Apache配置：/etc/apache2/
- 网站内容：/var/www/bank32/
- 证书：/certs/

为你的网站添加证书（实验任务四-2/3）



③ 部署新的证书文件（含私钥和证书）

将证书文件拷贝到共享目录，在**虚拟机**上执行

```
cp server.key Labsetup/volumes/  
cp server.crt Labsetup/volumes/
```

将证书文件转移到证书目录并重命名，在**容器**上执行

```
cp /volumes/server.key /certs/bank32.key  
cp /volumes/server.crt /certs/bank32.crt
```

④ 启动Apache服务器，在**容器**上执行

```
service apache2 start
```

将提示输入使用bank32.key中私钥的保护密码

⑤ 在**虚拟机**上修改/etc/hosts，配置域名访问

```
sudo vi /etc/hosts
```

在文件末尾添加如下配置

```
# For PKI Lab  
10.9.0.80      www.bank32.com
```



```
root@gef099bd1393:/certs# service apache2 start  
* Starting Apache httpd web server apache2  
Enter passphrase for SSL/TLS keys for www.bank32.com:443 (RSA):  
*
```

为你的网站添加证书（实验任务四-3/3）

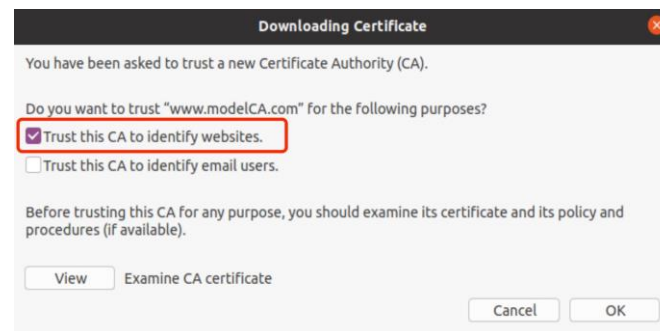
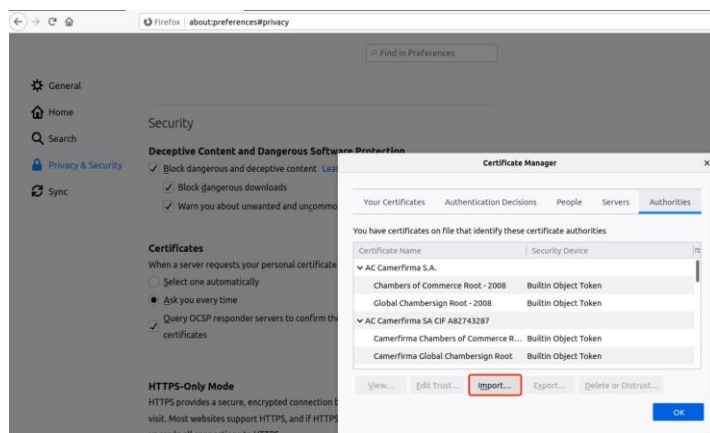


⑥ 打开浏览器，输入<https://www.bank32.com>查看结果



点击证书图标查看具体原因

⑦ 导入CA根证书，重新访问<https://www.bank32.com>查看结果

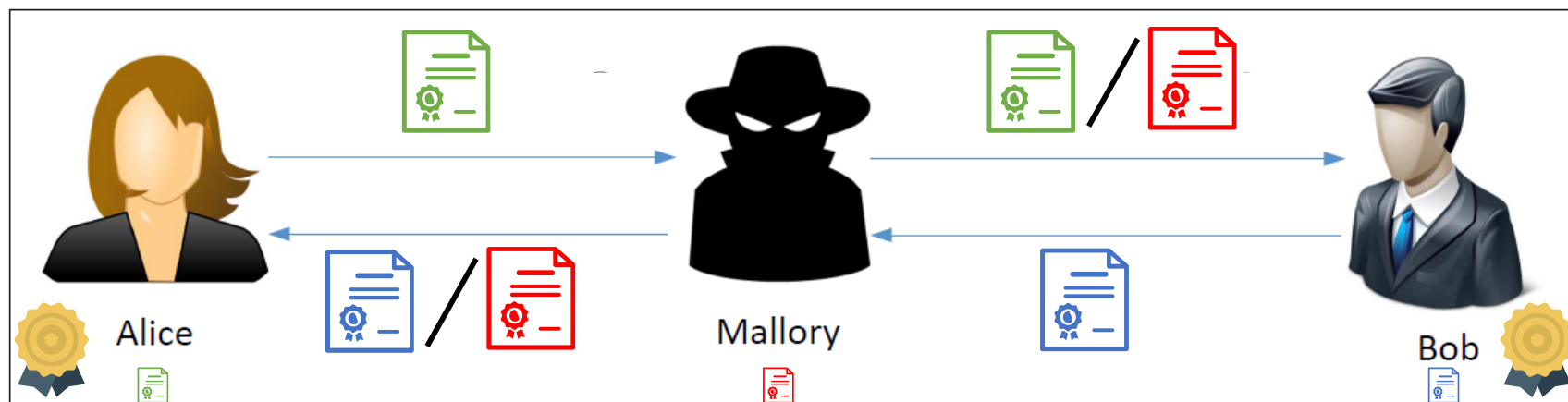


- 要求：记录程序、操作命令和输出结果，写入实验报告，对实验现象进行解释。

PKI 如何防御MITM攻击

➤ 攻击者（如Mallory）实施MITM的几种可能

- Mallory直接转发Alice的（真）证书
- Mallory伪造一个Alice的（假）证书
- Mallory发送自己的（真）证书



进一步思考：如果Mallory不是攻击者，而是被合法授权的？



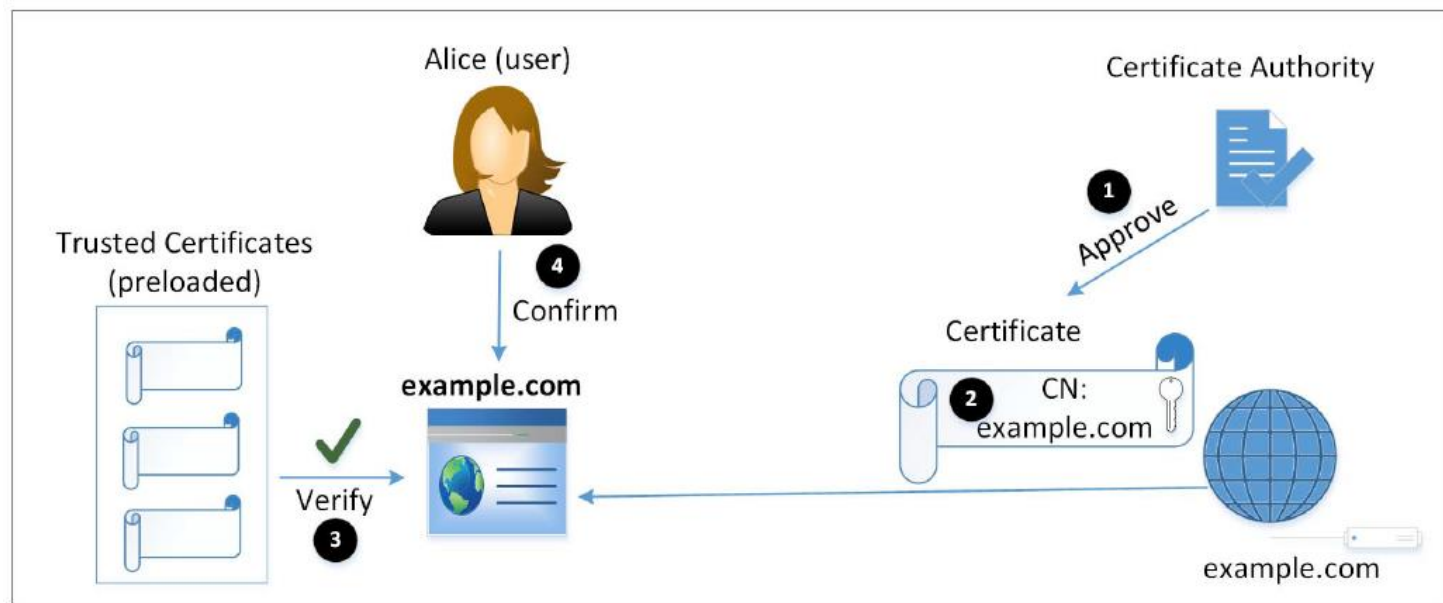
MITM Proxy

课程提纲

- 中间人（Man-In-The-Middle）攻击
- 公钥基础设施
- 数字证书：X. 509
- 如何获得和使用数字证书
- PKI 面临的安全风险



PKI 的攻击面



- ① CA验证阶段
- ② CA签名阶段
- ③ 证书验证阶段（密码学算法）
- ④ 用户确认阶段

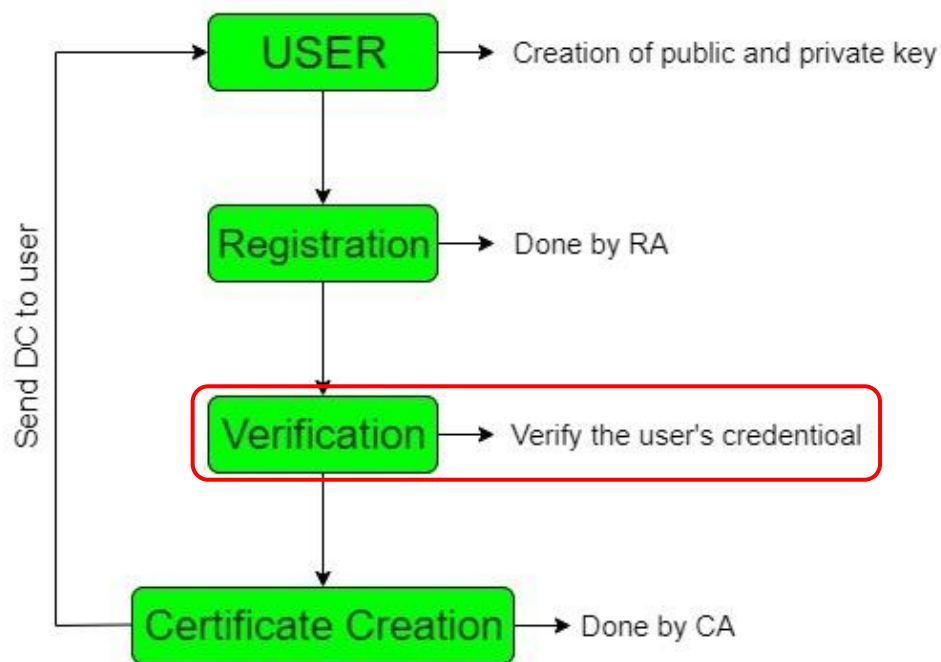
CA验证阶段

➤ CA的职责

- 验证证书请求者与其声称信息的关联性
 - 对证书进行数字签名
- DV: 邮件、服务器文件、DNS解析
 - OV/EV: 更严格的法律流程

➤ 案例: Comodo事件

- Comodo是一个知名的Root CA
- 欧洲南部分公司的CA验证服务器 (Registration Authority, RA) 被入侵
- 批准颁发了9个证书, 涉及到7个域名, 其中包含雅虎、火狐等知名企业



CA签名阶段

➤ CA的职责

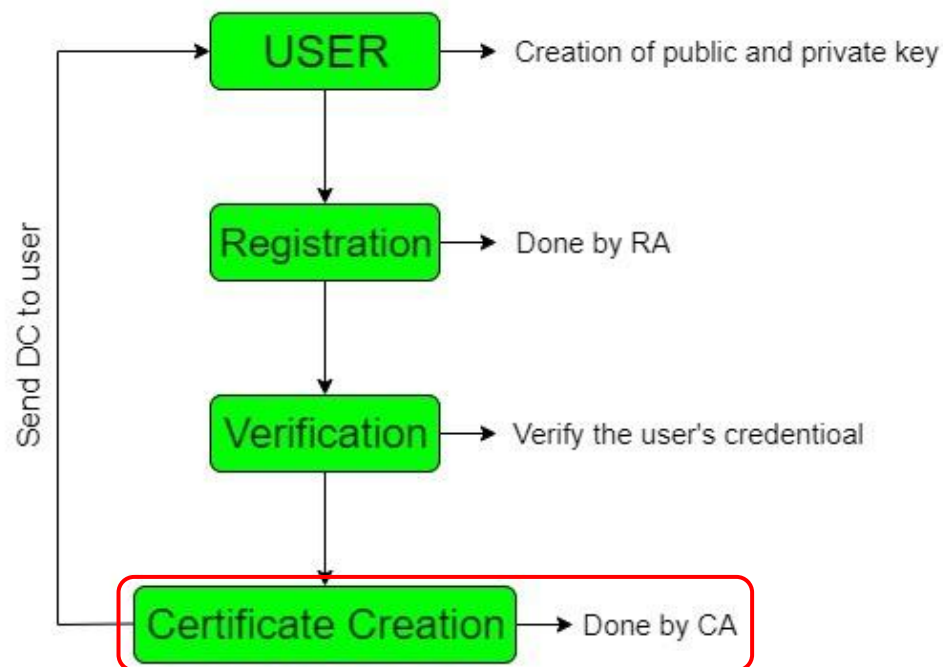
- 验证证书请求者与其声称信息的关联性
- 对证书进行数字签名（如果CA私钥被窃取？）

➤ 案例：DigiNotar事件

- DigiNotar是一家知名的商业CA
- 2011年其私钥被黑客窃取
- 签发了531份（合法的）虚假证书，包括Google在内的许多公司受到影响
- 数月后，DigiNotar宣告破产

➤ CA如何保护私钥

- 硬件安全模块（Hardware Security Module, HSM）
- 安保



Hardware Security Modules (HSMs)



证书中的密码学算法

➤ 证书中的密码学算法

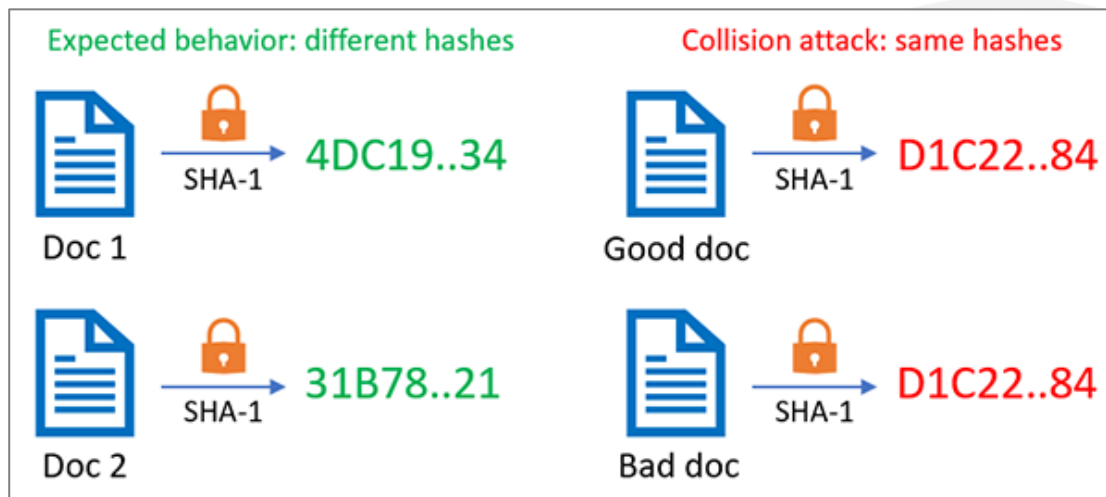
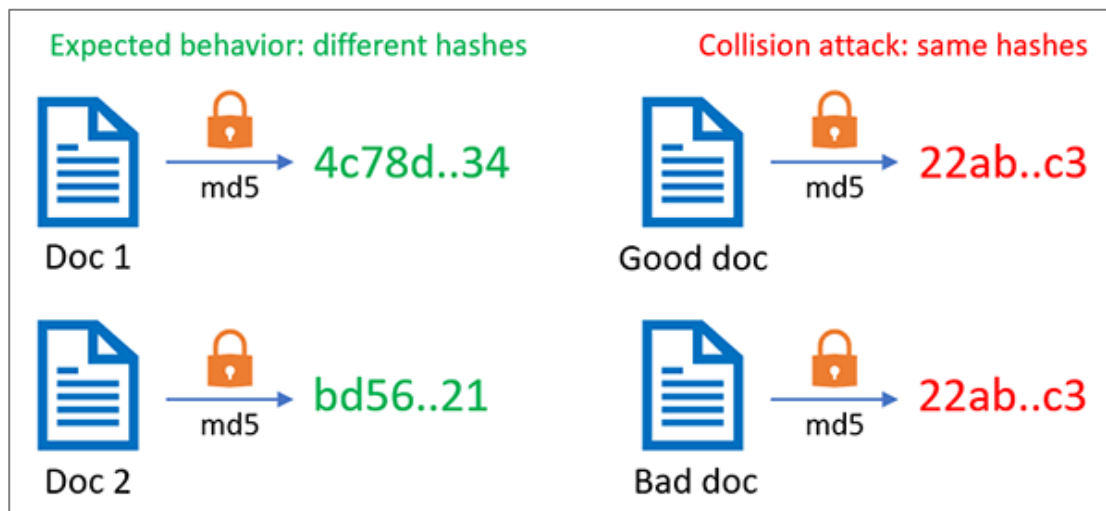
- 公钥加密：RSA，ECC
- 哈希：MD5、SHA

➤ 案例：哈希碰撞

- 在2004的密码学大会上，王小云院士团队展示了MD5的哈希碰撞
- 2017年，Google研究团队展示SHA1的哈希碰撞

➤ 如何防御

- 更健壮的密码学算法：如，SHA256



用户确认阶段

➤ 数字证书

- 解决了：用户ID与公钥的绑定关系
- 未解决：对用户ID的确认

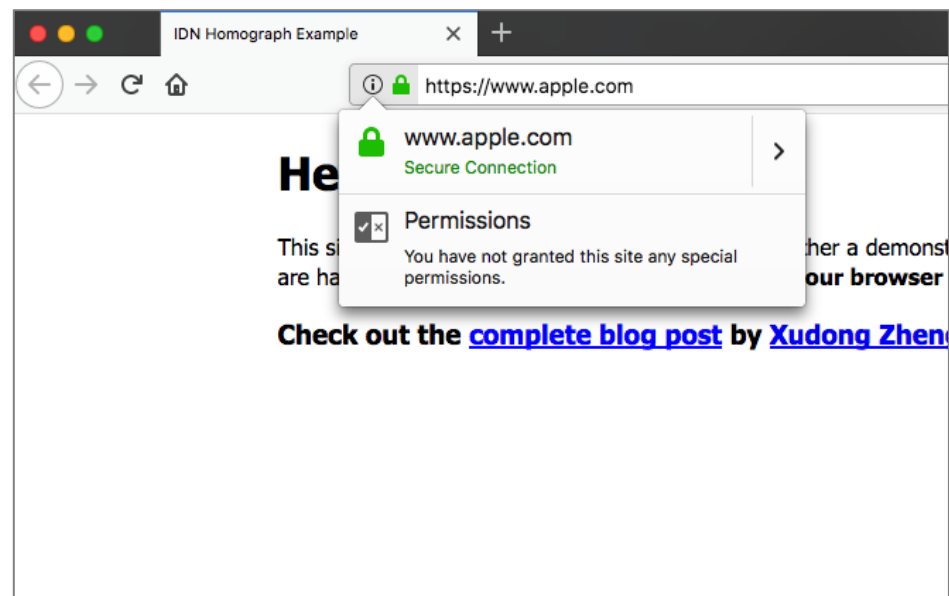
➤ 案例：利用Unicode编码进行钓鱼攻击

- 2017年，由Xudong Zheng提出
- 在国际化域名标签（RFC3492）的框架下，允许多语言域名（采用Unicode编码），显示时转换为ASCII码
- 许多Unicode字符看起来跟字母一样，但计算机却会处理成完全不一样网的网址。

➤ 如何防御

- 加强用户确认+浏览器辅助

斯拉夫字母 “a” （U+0430）和拉丁字母 “a” （U+0041）会被浏览器处理成不同的字符，但是在地址栏当中都显示为 “a”



xn-80ak6aa92e.com

小结

- 中间人攻击
- X.509证书、CA、数字签名
- 如何构建PKI体系
- PKI的安全问题

注意：为避免影响后续实验，请清除本实验用过的VM相关配置（恢复/etc/hosts，删除ModelCA根证书）